Short-Term Wind Speed Forecasting at Tehachapi, CA

Michel Faloughi, Alfred Hofmann, Gaoxiang Zhao

November 2025

Abstract

Accurate short-term wind speed forecasting is essential for reliable grid operation and efficient integration of renewable energy. The Tehachapi region in California, home to one of the largest wind farm clusters in the state, exhibits strong diurnal patterns and rapid short-term variability that challenge traditional forecasting approaches. In this project, we use three years of hourly ERA5 reanalysis data to evaluate several machine learning models for next-hour wind speed prediction at Tehachapi, including Linear Regression, Random Forests, XGBoost, and a small feed-forward neural network. In addition to standard mean squared error (MSE), we consider an asymmetric loss that penalizes underpredictions more heavily, reflecting real operational costs for wind farm operators.

Our results show that all machine learning methods outperform the persistence baseline, with XGBoost achieving the best overall accuracy. We further apply a post-hoc affine calibration that significantly reduces systematic underprediction across models and improves performance under the asymmetric loss. The calibrated neural network performs competitively with XGBoost when evaluated with asymmetric penalties. These findings highlight the value of combining reanalysis data with machine learning models and demonstrate that simple calibration techniques can meaningfully align model behavior with practical operational priorities in wind forecasting.

1 Motivation

California is rapidly expanding its renewable energy portfolio, making accurate short-term forecasting essential for reliable grid operation. The Tehachapi region, home to one of the largest wind farm clusters in the state, is a natural testbed for studying wind variability and improving prediction tools.

Short-term wind speed forecasting matters because wind conditions change quickly, and even small errors can affect reserve planning, dispatch decisions, and overall grid stability. Better forecasts also support broader atmospheric monitoring and early warning for extreme weather, such as hurricanes.

Traditional numerical weather prediction models are computationally intensive and often struggle with site-specific, hour-ahead accuracy. At the same time, modern atmospheric datasets such as ERA5 are large, high-resolution, and well suited for data-driven methods. This makes machine learning an attractive alternative: ML can model nonlinear relationships among many atmospheric variables and provide fast, flexible predictions.

In practice, engineers at the Tehachapi wind farm are especially sensitive to *underpredicting* wind speed, as it leads to committing too little generation, insufficient reserves, and unnecessary corrective actions. These errors increase operational costs and reduce the use of available renewable energy. This motivates our interest in models that specifically discourage underestimation. We therefore explore machine learning approaches that penalize underprediction, aligning the forecasting objective with real operational needs at Tehachapi.

Our goal in this project is to evaluate several machine learning approaches for next-hour wind speed forecasting at Tehachapi and to design models that perform better in operational settings by reducing harmful underpredictions.

Our full codebase and reproducible experiments are available at: `https://github.com/MichelFaloughi/cis5200-project`.

2 Related Work

Wind speed forecasting has been studied extensively using both statistical and machine learning approaches. Early work, such as Mohandes et al. [6], showed that neural networks could capture nonlinear patterns in wind speed time series, though these models were limited by small and low-resolution datasets.

The introduction of high-quality reanalysis products like ERA5 [4] enabled more reliable data-driven forecasting. Liu et al. [5] used ERA5 variables with boosted tree models for short-term wind prediction and showed that ensemble methods can substantially outperform simple statistical baselines. However, their work does not address the operational consequences of underprediction, which is a key concern in wind farm management.

Ensemble tree methods such as Random Forests [1] and Gradient Boosting Machines [2] remain among the strongest classical models for nonlinear regression and form part of our comparison. Work by Gneiting et al. [3] highlights the importance of aligning forecasting objectives with real-world decision costs, motivating our use of asymmetric losses that penalize underestimation.

Prior research demonstrates the effectiveness of ML models for short-term wind forecasting and the value of rich atmospheric datasets like ERA5. Our work builds on these ideas by comparing several ML approaches for next-hour prediction at Tehachapi and by explicitly targeting the reduction of harmful underpredictions.

3 Dataset

Our analysis uses hourly meteorological data from the ERA5 global reanalysis [4], a high-resolution atmospheric dataset produced by the European Centre for Medium-Range Weather Forecasts (ECMWF). We extract variables for the Tehachapi region, centered at approximately 35.10° N and 118.30° W, from January 2018 through December 2020, yielding a total of $n = 26{,}303$ hourly samples with $p = 13$ predictor features.

The dataset includes the 10 m horizontal wind components $u10$ and $v10$, 2 m air temperature $t2m$, and surface pressure sp. To capture diurnal, monthly, and seasonal cycles, we encode time using standard cyclical transformations. For any periodic quantity x with period P, we define

$$x_{\sin} = \sin\left(2\pi \frac{x}{P}\right), \qquad x_{\cos} = \cos\left(2\pi \frac{x}{P}\right).$$

We apply this encoding to the hour of day ($P = 24$), month of year ($P = 12$), and day of year ($P = 365$), producing the features `hour_sin`, `hour_cos`, `month_sin`, `month_cos`, `doy_sin`, and `doy_cos`. These encodings map each temporal variable onto the unit circle, allowing the models to learn seasonal and diurnal patterns without discontinuities.

We compute the scalar wind speed at time t as

$$\text{wind_speed}_t = \sqrt{u10_t^2 + v10_t^2},$$

and define the forecasting target as the wind speed one hour ahead, $y_t = \text{wind_speed}_{t+1}$. Shifting the target introduces a single missing value, which we remove before training.

Figure 1 (right panel) shows the distribution of wind speed across the three-year period. The distribution is right-skewed, with most values between 1–4 m/s but with occasional higher-wind events up to 9–10 m/s. A two-week segment of the time series (Figure 2) highlights the strong diurnal structure characteristic of the Tehachapi region.

Figure 1 (left panel) summarizes several key variables. Wind speed ranges from 0.03 m/s to 9.21 m/s, with a mean of 1.50 m/s and substantial seasonal variability. The meteorological variables span realistic ranges for this region, and the cyclical time features each lie in $[-1, 1]$ by construction.

We perform minimal preprocessing beyond feature construction and target shifting. Because tree-based models do not require feature scaling, we apply no normalization. The dataset is split chronologically into training, validation, and test sets to respect the temporal ordering inherent in the forecasting task.

Variable	Mean	Std	Min	Max
$u10$	1.70	2.43	-8.16	9.21
$v10$	-0.24	1.47	-5.59	6.98
$t2m$ (K)	289.43	9.03	267.32	313.32
sp (Pa)	89768	384	88059	91198
wind_speed	1.51	0.71	0.03	9.21

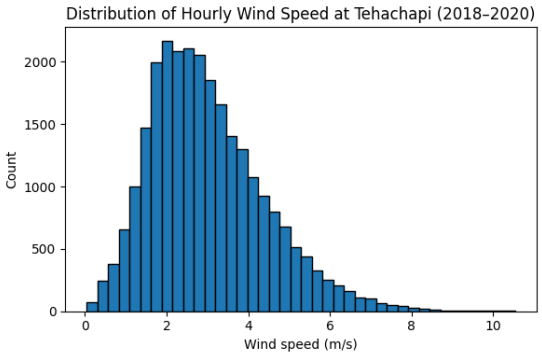


Figure 1: Left: summary statistics for selected predictors. Right: histogram of hourly wind speed (2018–2020).

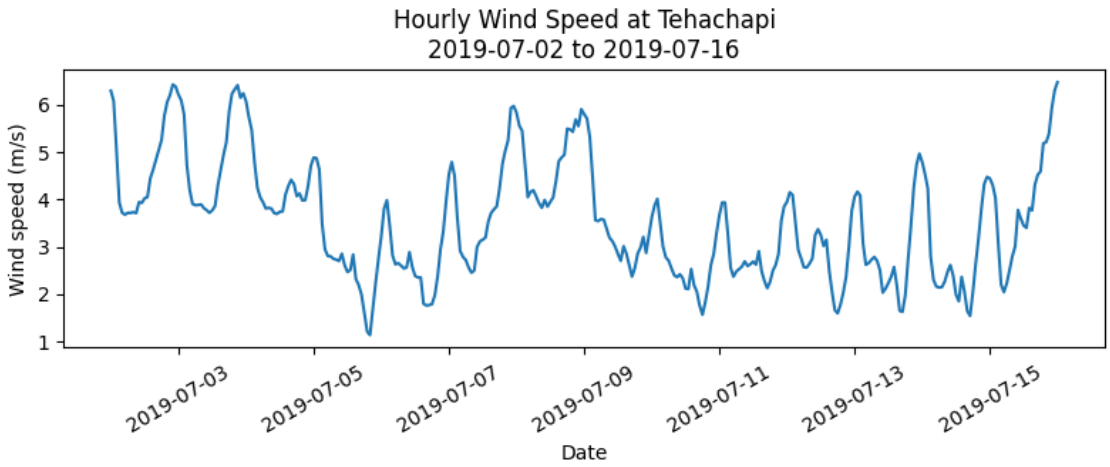


Figure 2: Example two-week period of hourly wind speed, illustrating strong diurnal oscillations and short-term variability typical of the Tehachapi region.

4 Problem Formulation

We frame wind speed forecasting as a supervised regression task in which each example contains meteorological and time-based predictor variables, and the goal is to estimate wind speed one hour ahead. Regression models are well suited for this continuous prediction problem, and we compare simple linear methods with more flexible nonlinear models. Cyclical encodings of hour, month, and day-of-year help capture periodic patterns that influence wind behavior.

Most models in our study like Linear Regression, Random Forest, XGBoost, and the Persistence baseline are trained using mean squared error (MSE). MSE is a standard regression objective that penalizes large deviations and is compatible with the optimization procedures used in these classical models. However, MSE does not reflect an important real-world asymmetry: in grid operations, underpredicting wind availability is more costly than overpredicting, because low forecasts can trigger unnecessary reserve generation or inefficient dispatch decisions.

To incorporate this asymmetry, we train our neural network using a custom loss function that penalizes underestimation more heavily than overestimation. Neural networks support gradient-based optimization, allowing us to directly encode operational priorities into the learning objective and reduce systematic underforecasting.

$$L(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^{n} w_i \, (\hat{y}_i - y_i)^2, \quad \text{where} \quad w_i = \begin{cases} \alpha & \text{if } \hat{y}_i < y_i, \\ 1 & \text{otherwise.} \end{cases}$$

Traditional models cannot be trained end-to-end on arbitrary custom losses, so we instead apply a post-hoc asymmetric calibration. After training each model with MSE, we learn a simple affine transformation of its outputs that minimizes the same asymmetric loss used for the neural network. This allows all forecasting methods to align with the domain objective (avoiding costly underpredictions) while retaining their native training procedures.

5 Methods

5.1 Baseline Model

As a reference point, we use a persistence baseline, which predicts that the wind speed one hour ahead will be equal to the current wind speed. This method does not require training and reflects the common short-term behavior of wind patterns, where changes often occur gradually rather than abruptly. We chose this baseline because it represents a simple, physically meaningful strategy that is commonly used in operational forecasting as a minimum performance standard. Any machine learning model must outperform this baseline to demonstrate that it captures useful structure beyond what is already inherent in the data.

5.2 Linear Regression

For our first learning-based model, we use regularized linear regression in a pipeline with polynomial feature expansion and feature scaling. We treat this as a simple but interpretable model that can capture simple nonlinear relationships while remaining easy to analyze. To respect the temporal structure of the data and avoid leakage from the future into the past, we use a time-series aware cross-validation scheme (TimeSeriesSplit with five splits) instead of random shuffling. The pipeline first generates polynomial features of degree one or two, then standardizes all inputs so that the regularization term in Ridge regression acts fairly across coefficients. We use Ridge rather than ordinary least squares because our features (and their polynomial interactions) have some correlation as revealed by a heatmap, and L2 regularization helps control multicollinearity and reduce overfitting. Hyperparameters for the polynomial degree and the Ridge penalty strength are selected via GridSearchCV, using negative mean squared error as the scoring function to align with our RMSE-based evaluation. After model selection, we refit the best pipeline on the earliest 80 percent of the data and evaluate on the final 20 percent, preserving chronological order to obtain an unbiased estimate of out-of-sample performance using RMSE and R^2.

5.3 Random Forest

After experimenting with the linear model, we use a Random Forest regressor to better capture nonlinear relationships between weather variables and future wind speed. We find from prior research that Random Forests are well suited for environmental prediction tasks because they can model complex interactions without requiring heavy feature engineering, and they are robust to noise and correlated inputs. To preserve the temporal structure of the data, we again use a chronological 80/20 split for testing and apply TimeSeriesSplit only on the training portion. We begin by evaluating a reasonable default configuration to establish how the model performs without tuning, and then refine it using a randomized hyperparameter search focused on the most influential settings, such as the number of trees, maximum depth, split criteria, and feature selection strategy. This approach allows the model to balance flexibility and generalization while remaining computationally efficient. The final tuned forest is refit on the full training period and evaluated against the held-out test window to provide an unbiased assessment of its forecasting ability.

5.4 Gradient Boosting

We utilized Extreme Gradient Boosting (XGBoost), which ultimately yielded the strongest predictive performance among all evaluated models. In contrast to the parallel bootstrapping approach of Random Forests, XGBoost builds an ensemble of decision trees sequentially. Each new tree is trained to predict the residual errors of the preceding ensemble, effectively performing gradient descent in function space to minimize the

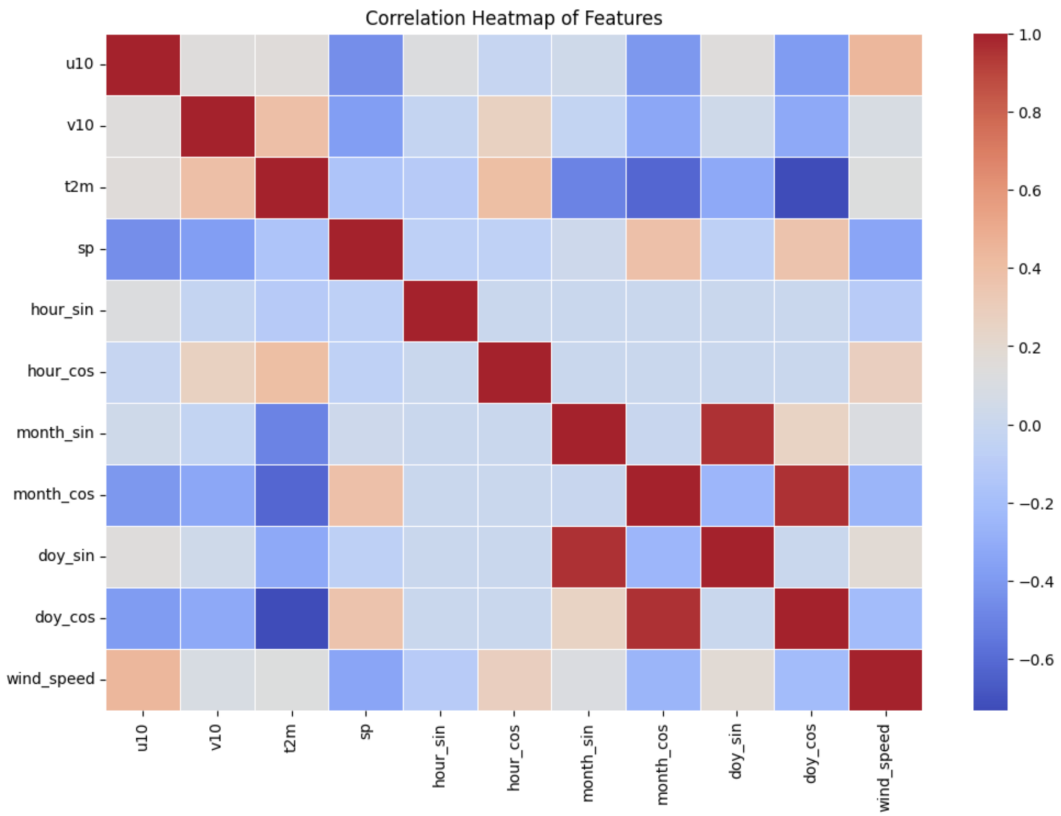


Figure 3: Heatmap of features and target correlation.

loss function. Through hyperparameter tuning, we selected a configuration with 300 estimators, a learning rate of 0.05, and a maximum tree depth of 6 to balance model complexity with generalization capability.

This boosting framework proved particularly effective at capturing the sharp fluctuations and complex non-linear patterns inherent in the wind speed data. By iteratively refining predictions, the model progressively reduces bias and focuses computational resources on difficult-to-predict samples. Furthermore, the algorithm incorporates regularization techniques and stochastic subsampling, which help prevent overfitting even when modeling intricate dependencies. This combination allows XGBoost to produce highly accurate estimates that significantly outperform both the linear baselines and other ensemble methods.

5.5 Multilayer Perceptron

To capture continuous non-linear dependencies between meteorological variables and future wind speed, we implemented a Feedforward Multilayer Perceptron (MLP). Unlike the tree-based ensemble methods, which require post-hoc calibration to address our specific operational goals, the MLP is fully differentiable. This unique property allowed us to incorporate domain knowledge directly into the learning objective. Instead of minimizing the standard Mean Squared Error, we trained the network end-to-end using the custom asymmetric loss function defined in the Problem Formulation (Section 4). This ensures that the learned feature representations are inherently aligned with the goal of minimizing costly underpredictions.

Prior to training, we standardized all input features to ensure zero mean and unit variance, a critical step for the convergence of gradient-based optimization. The network was optimized using the Adam solver with a learning rate of 1×10^{-3} for 500 epochs.

5.5.1 Architecture and Hyperparameter Selection

Defining the optimal network topology for tabular wind data is non-trivial. To select a robust architecture, we conducted extensive comparative experiments focusing on hidden layer dimensions and activation functions.

Hidden Layer Dimensions We evaluated architectures with varying layer widths, ranging from highly constrained configurations (e.g., [4, 2]) to wide, over-parameterized networks (e.g., [200, 100]). Experimental results demonstrated that performance did not scale linearly with model capacity. Larger configurations exhibited higher errors, likely due to overfitting the finite training data, while extremely narrow networks lacked the representational power to capture the signal. We identified the configuration of two hidden layers with **8 and 4 neurons** respectively as the optimal balance. This compact architecture achieved competitive accuracy while maintaining low model complexity, suggesting that the underlying predictive signal in this dataset is relatively low-dimensional and best modeled without excessive parameters.

Activation Functions We further validated our choice of non-linearity by comparing Sigmoid, Tanh, Leaky ReLU, and ReLU activations on the fixed (8, 4) architecture. Our results indicated that **ReLU** consistently outperformed saturating activation functions like Sigmoid and Tanh. Theoretically, ReLU mitigates the vanishing gradient problem, allowing for more efficient weight updates during backpropagation. Furthermore, the sparsity induced by ReLU is well-suited for modeling the sharp, threshold-like transitions often observed in wind speed fluctuations. Based on these analyses, our final deployed MLP utilizes the (8, 4) architecture with ReLU activations.

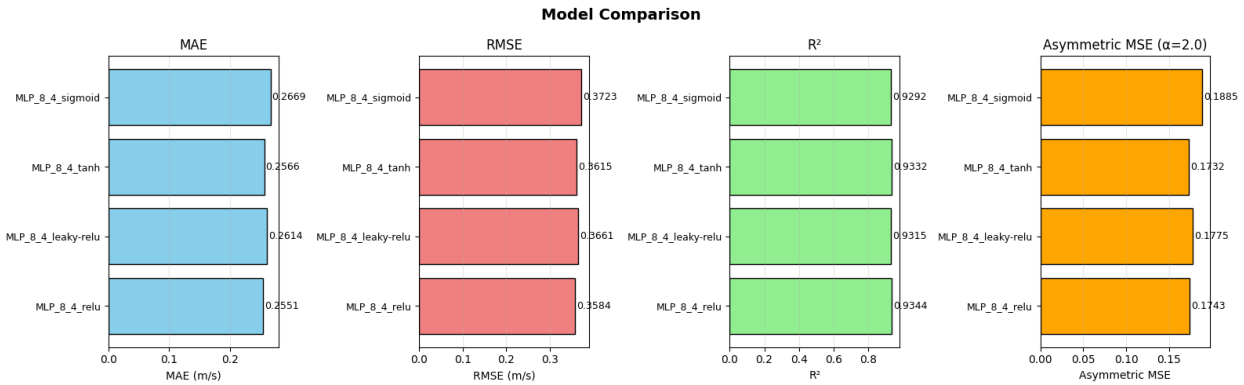


Figure 4: Performance comparison of different activation functions in the MLP. ReLU generally yields the lowest errors, justifying its selection for the final model comparison.

6 Experiment and Results

For the non-neural models, we first compare performance using standard metrics before applying the asymmetric transformation. This provides an initial sense of each model's predictive capability under conventional evaluation criteria and helps ensure that the models behave sensibly prior to calibration. We do this because models that perform poorly under standard losses are unlikely to improve meaningfully under the custom loss, and strong baseline accuracy generally indicates that a model will benefit more from post-hoc adjustment. Based on these initial comparisons, XGBoost achieves the highest accuracy, and all models outperform the persistence baseline.

After training each model on standard mean squared error, we apply a post-hoc transformation to align their predictions with our asymmetric operational objective. For the non-neural models, this involves learning a simple affine calibration layer $y' = a\, y_{\text{pred}} + b$ that is optimized on a validation set to minimize the same custom loss that penalizes underprediction more heavily than overprediction. This transformation effectively corrects systematic low-bias tendencies without modifying the underlying models or retraining them, allowing each predictor to produce outputs that better reflect the real-world costs of underestimating wind availability, regardless of its native loss function.

After establishing baseline performance, we apply our post-hoc asymmetric calibration to each non-neural model so that their final predictions better reflect the operational cost structure encoded in our custom loss. This calibration consistently improved asymmetric loss across all models, showing that predictors trained with standard MSE can still be adjusted effectively to reduce costly underpredictions. All calibrated models performed better than the persistence baseline. The neural network, which was trained directly on the custom asymmetric loss, achieved the second-best performance among all models on the custom loss aside from XGBoost. XGBoost slightly outperformed it even though it was trained with standard loss and only calibrated afterward because XGBoost captures nonlinear relationships and threshold effects very effectively, producing highly accurate base predictions. Once these predictions are shifted through the asymmetric affine transformation, XGBoost combines strong initial accuracy with alignment to the custom loss, allowing it to edge out the neural network.



Figure 5: Model Comparison. The bar charts display performance across MAE, RMSE, R^2, and the custom Asymmetric MSE ($\alpha = 2.0$). XGBoost achieves the lowest error and highest R^2, while also minimizing the asymmetric operational cost.

7 Conclusion and Discussion

In this project, we evaluated several machine learning models for one-hour-ahead wind speed forecasting at the Tehachapi wind farm using ERA5 reanalysis data. Our models included Linear Regression, Random Forests, XGBoost, and a feed-forward neural network. Across all standard metrics, XGBoost achieved the strongest overall performance, with the lowest MAE and RMSE and the highest R^2. All models substantially outperformed the persistence baseline. After applying our asymmetric loss, which penalizes underpredictions more heavily, we found that a simple post-hoc affine calibration step consistently improved alignment with the operational objective for every non-neural model. Under this asymmetric evaluation, XGBoost remained the top performer, followed by the neural network trained directly with the custom loss.

Qualitatively, the results reinforce several insights into short-term wind forecasting. First, ERA5 reanal-

ysis contains enough signal at hourly resolution for data-driven models to capture strong diurnal structure and short-term fluctuations. Second, most models exhibited a natural tendency to slightly underpredict wind speed, likely due to right-skewed wind distributions and the greater frequency of low-to-moderate wind events. This highlights the practical value of including an asymmetric loss or calibration procedure when underprediction carries higher operational costs. Third, the effectiveness of a simple affine correction indicates that a large portion of the underprediction came from a systematic global bias rather than from complex nonlinear mismatches.

During the project, we identified several lessons about the modeling process. Using cyclical time encodings (hour, month, day-of-year) was essential, and models performed noticeably worse without these features. Feature scaling was unnecessary for tree-based methods but important for stabilizing neural network training. We also learned that tuning the asymmetric loss directly inside the neural network was effective, while attempting to train tree-based models directly under the asymmetric objective was unstable, which motivated the use of post-hoc calibration. Additionally, overly complex architectures did not improve performance, and simple models combined with calibration performed best.

There are several promising directions for future extensions. A natural next step is to explore multi-step forecasting (for example, 3 to 6 hour horizons) or probabilistic forecasts that provide uncertainty intervals. More expressive temporal models such as LSTMs, TCNs, or Transformers may better capture short-term dynamics, while spatial models that incorporate neighboring ERA5 grid points could improve accuracy in terrain-influenced regions like Tehachapi. Finally, integrating these forecasts into a simulated operational or energy market environment would help quantify their real-world economic impact, particularly in settings where underprediction is costly.

Overall, our findings show that relatively simple machine learning approaches, when combined with appropriate temporal encodings and post-hoc calibration, can deliver accurate and operationally aligned wind forecasts for the Tehachapi region.

References

[1] Leo Breiman. Random forests. *Machine learning*, 2001.

[2] Jerome H Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 2001.

[3] Tilmann Gneiting, Kristin Larson, Kenneth Westrick, Marc G Genton, and Eric Aldrich. Calibrated Probabilistic Forecasting at the Stateline Wind Energy Center. *Journal of the American Statistical Association*, 2006.

[4] Hans Hersbach, Bill Bell, Paul Berrisford, Shoji Hirahara, András Horányi, Joaquín Muñoz-Sabater, Julien Nicolas, et al. The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 2020.

[5] Y. Liu, Z. Wang, and H. Zhang. A hybrid machine learning framework for short-term wind speed forecasting using ERA5 reanalysis data. *Renewable Energy*, 2023.

[6] M.A. Mohandes, S. Rehman, and T.O. Halawani. A neural networks approach for wind speed prediction. *Renewable Energy*, 1998.